

“云计算安全” 课程论文阅读报告

题目: Visor: Privacy-Preserving Video Analytics as a
Cloud Service

班级:	<u>硕 2505</u>
姓名:	<u>叶一璇</u>
学号:	<u>M202572310</u>

2025 年 11 月 18 日

摘要

视频分析即服务（Video-analytics-as-a-service）正成为云提供商的一项重要业务。此类服务的核心关注点是被分析视频的隐私性。虽然可信执行环境（TEEs）是防止私有视频内容直接泄露的有前景的选项，但它们仍然容易受到侧信道攻击的威胁。本文提出了 **Visor**，这是一个即使在云平台受损且存在不可信共租户的情况下，也能为用户的视频流以及机器学习模型提供机密性的系统。**Visor** 在一个跨越 CPU 和 GPU 的混合 TEE 中执行视频管道。它保护管道免受由视频模块的数据依赖型访问模式引发的侧信道攻击，并解决了 CPU-GPU 通信通道中的泄露问题。**Visor** 比朴素的数据模糊解决方案快高达 1000 倍，且相对于非模糊基线的开销限制 2 至 6 倍之间。

关键词：可信执行环境（TEE）；视频分析；侧信道攻击；数据模糊性；隐私保护

目录

摘要.....2

1.绪论.....4

 1.1 研究背景.....4

 1.2 研究现状.....4

 1.3 关键概念.....5

2.研究内容.....6

 2.1.创新点.....6

 2.2.系统架构.....6

 2.3.关键技术.....7

3.总结.....9

 3.1.局限性.....9

 3.2 改进思路.....9

 3.2.总结.....10

参考文献.....11

1.绪论

1.1 研究背景

随着物联网和智慧城市的发展，摄像头的部署日益普及，被广泛应用于交通规划、零售体验优化以及企业安防等领域。由此产生了海量的视频数据，这些数据通常被流式传输到云端，通过由计算机视觉技术和卷积神经网络组成的视频分析管道进行处理。这种“视频分析即服务”模式已成为云服务提供商的重要产品。

然而，视频内容的隐私性成为了该服务模式下的首要关注点。视频通常包含高度敏感的信息，例如用户的家庭内部情况、工作场所的人员活动，或者是车辆牌照等。为了保护用户隐私，视频流必须保持机密，不得泄露给云服务提供商或云环境中的其他共租户。例如，攻击者若能推断出视频内容，可能会分析出居民在家的作息时间和医疗中心的患者访问记录，导致严重的隐私侵犯。因此，如何在利用云端强大算力的同时，确保视频数据在处理全流程中的机密性，是当前亟待解决的问题。

1.2 研究现状

目前，保护云端数据隐私的主流技术路线主要分为基于密码学的方法和基于硬件的可信执行环境。

1.密码学方法：如同态加密和安全多方计算。虽然这些方法在理论上能提供较强的安全性，但在处理复杂的视频分析任务（尤其是涉及深度神经网络时）效率极低，往往比明文执行慢数个数量级，难以满足实时视频分析的需求。

2.可信执行环境：如 Intel SGX 和 ARM Trust Zone。TEE 允许在不可信的硬件上创建受保护的“飞地”（Enclave），即便是拥有特权的操作系统或 Hypervisor 也无法直接访问其中的内存数据。由于 TEE 的性能损耗远低于密码学方法，它被视为隐私保护云计算的核心技术。随着 GPU TEE 的出现，在 TEE 中运行高负载的 ML 模型也成为可能。

现有挑战：尽管 TEE 能防止内存数据的直接读取，但它们对侧信道攻击非常脆弱。攻击者可以通过观察程序的内存访问模式来推断受保护的数据内容。

例如，在执行物体检测算法时，程序访问内存的轨迹往往与图像中物体的形状和位置高度相关。现有的通用数据模糊执行系统（如 ORAM）虽然能掩盖访问模式，但应用于复杂的视频分析管道时会带来难以接受的性能开销（通常慢 10^6 到 10^7 倍）。因此，缺乏一种既能防御侧信道攻击，又能保持高性能的专用视频分析隐私保护系统。

1.3 关键概念

为了深入理解本文的研究内容，需要明确以下几个关键概念：

- **可信执行环境：**一种基于硬件的安全机制，保护代码和数据免受系统其他软件（包括 OS）的干扰和窥探。Intel SGX 是 CPU TEE 的典型代表，通过将代码和数据存储在受保护的内存区域来强制隔离。
- **侧信道攻击：**攻击者不直接破解加密算法，而是利用系统在处理数据时产生的物理特征（如时间延迟、功耗、电磁辐射或内存访问模式）来泄露信息。本文主要关注基于数据依赖型内存访问模式的微架构侧信道攻击。
- **数据模糊性：**指程序的执行路径和内存访问序列完全独立于输入的数据内容。无论输入图像是全黑还是包含复杂的交通场景，数据模糊算法产生的内存访问痕迹对攻击者来说都是不可区分的。
- **混合 TEE 架构：**指结合了 CPU TEE（用于逻辑控制和轻量级预处理）和 GPU TEE（用于高强度并行计算，如 CNN 推理）的安全架构。Visor 利用这种架构来平衡安全性和性能。

2. 研究内容

本文提出了 Visor 系统，旨在解决云端视频分析中的侧信道泄露问题。

Visor 不仅仅是一个简单的工具堆叠，而是对视频分析管道中的核心算法进行了彻底的重构。

2.1. 创新点

Visor 的主要贡献体现在以下几个方面：

1. 隐私保护的 MLaaS 框架：提出了一种跨越 CPU 和 GPU 资源的混合 TEE 框架。该框架不仅保护视频流，还保护机器学习模型参数。它通过安全协议连接 CPU 和 GPU，确保了两者之间的数据传输不会通过流量模式泄露信息。

2. 高效的数据模糊视觉算法：Visor 没有采用通用的 ORAM 方案，而是针对视频分析中常用的视觉模块（如解码、背景扣除、包围盒检测、裁剪等），设计了定制化的数据模糊算法。这些算法在保证数学上的隐私可证明性的同时，性能比朴素的模糊实现快了 1000 倍。

3. 算法设计原则的提炼：作者总结了一套通用的模糊算法设计策略，包括“分而治之”、“基于扫描的顺序处理”以及“跨像素组的成本摊销”。这些原则可以指导未来其他视觉模块的隐私保护设计。

4. 端到端的隐私保护：除了计算过程的防护，Visor 还考虑了网络传输层面的泄露，通过修改视频编码器在源端进行填充，防止基于比特率变化的流量分析攻击。

2.2. 系统架构

Visor 采用了混合 TEE 架构，如图 1 所示，系统主要由以下部分组成：

CPU TEE (基于 Intel SGX)：负责运行非 CNN 的轻量级视频处理模块，包括视频解码、背景扣除、物体检测和裁剪；运行 Graviton 的 GPU 运行时，负责安全地引导 GPU TEE 并建立信任域；处理应用逻辑，并在将数据发送给 GPU 之前进行预处理。

GPU TEE (基于 Graviton)：负责运行计算密集型的 CNN 分类器；通过安全的命令处理器接收来自 CPU 的加密指令和数据，确保 GPU 上的计算对外界不

可见。

安全的 CPU-GPU 通信通道：数据在 CPU 和 GPU 之间传输时是加密的，为了防止流量模式泄露（即通过传输的数据量推断每帧中的物体数量），Visor 引入了“循环对象缓冲区”。CPU TEE 每帧提取固定数量的对象，不足部分用哑元/dummy objects 填充，并以固定速率发送给 GPU。另外，为了减少 GPU 处理哑元对象的浪费，Visor 设计了一种模糊排序协议，将真实对象排在缓冲区尾部优先处理，尽量覆盖掉头部的哑元对象。

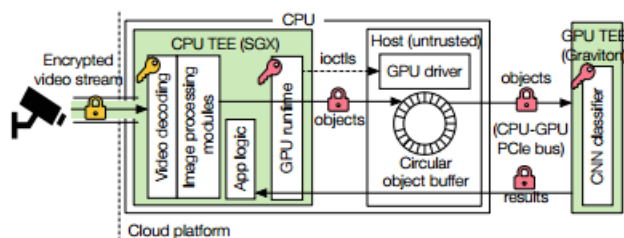


图 1 Visor 混合 TEE 架构

2.3.关键技术

Visor 的核心在于其一系列经过重构的数据模糊算法，这些算法利用基础的模糊原语构建。

模糊视频解码视频解码通常包含大量的条件分支和查表操作，极易泄露信息。传统的霍夫曼树或算术解码依赖于输入比特流进行树的遍历，遍历路径直接泄露了系数值。采用模糊前缀树遍历，将前缀树展平为数组，使用 `oaccess` 进行访问。为了隐藏当前是否到达叶子节点，Visor 引入了哑元节点，并强制解码器在每一步都输出一个值，使得外界无法区分正在处理的是树的哪个部分。另外，解码器不直接将系数写入像素位置（这会导致随机写访问），而是先将解码出的系数存入列表，然后基于元组进行模糊排序，将真实系数按像素顺序排列，最后顺序写入图像。为了进一步优化 `osort` 的性能（排序复杂度是超线性的），Visor 不在帧级别进行处理，而是以“块行”为粒度进行填充和解码，显著降低了排序的数据规模。

模糊背景扣除：传统算法（如高斯混合模型 **GMM**）对每个像素维护不同数量的高斯分量，根据像素值的稳定性动态增删分量，这种差异化的处理流程会泄露像素的变化情况（即物体运动轨迹）。采取强制每个像素始终维护固定数量的高斯分量。对于不足的部分，使用权重为 0 的哑元分量填充。无论像素值

如何，系统都执行相同的更新操作。由于操作整齐划一，这还使得利用 SIMD 指令进行并行加速成为可能。

模糊包围盒检测：常用的边界跟随算法会沿着物体边缘扫描，其内存访问轨迹直接描绘了物体的形状。Visor 采用了两阶段的 CCL 算法。第一阶段扫描图像分配临时标签，第二阶段合并标签。通过限制临时标签的最大数量，并结合 oassign 和 osort，使得算法的执行路径与图像内容无关。另外，Visor 将图像水平切分为多个条带，并行处理每个条带内的包围盒，最后再合并跨条带的边界。这不仅提高了并行度，还减小了每个子任务的数据规模，大幅降低了算法延迟。

模糊物体裁剪与缩放：如果直接根据计算出的坐标去复制像素，内存访问地址会泄露物体的位置和大小。朴素的滑动窗口方法会产生大量冗余拷贝，导致 4000 倍的性能下降。为此，Visor 将裁剪过程分解为水平和垂直两个阶段。第一阶段只在垂直方向滑动，提取包含物体的水平条带；第二阶段在水平条带内滑动，提取最终物体。这种降维打击消除了大部分冗余操作。为了隐藏物体的真实大小，Visor 将所有裁剪出的物体缩放到一个固定的上限尺寸。为了利用缓存局部性，在进行第二阶段插值前，会对图像进行转置。

CNN 推理的模糊性：在 GPU 上运行的 CNN 主要由矩阵乘法组成，其访问模式本质上是数据无关的。对于 ReLU 和 Max Pooling 等条件操作，Visor 利用 GPU 的无分支指令来实现，从而天然保证了推理过程的模糊性。

3.总结

3.1.局限性

尽管 Visor 在性能和安全性上取得了平衡，但阅读报告中仍需指出其潜在的问题：

1. 公共参数的依赖：Visor 的高效性在很大程度上依赖于预设的上限参数。这些参数被视为公共信息。如果攻击者知晓这些上限，可能会构造对抗性输入，例如制造包含大量微小物体或超过上限的物体数量的视频帧，导致系统处理溢出或被迫进入低效的 Worst-case 处理路径，从而可能引发拒绝服务攻击或泄露特定帧的异常状态。

2. 资源开销增加：为了维持模糊性，Visor 采用了大量的填充和哑元计算。视频流的行级填充增加了网络带宽消耗（约 40%-50% 的增长）。GPU 为了处理哑元对象，虽然通过优化算法减少了浪费，但在低负载下仍然比原生系统消耗更多的计算资源。

3. 硬件依赖性：Visor 目前依赖于特定的 SGX 和 Graviton 硬件特性。SGX 本身存在的某些侧信道漏洞虽然在 Visor 的威胁模型之外，但在实际部署中仍是风险点。此外，要求关闭超线程以防止同核侧信道攻击，这会直接导致约 30% 的并发性能损失。

4. 遗留系统的兼容性：Visor 要求在摄像头源端就进行特殊的比特流填充。然而，大量现存的遗留 Legacy cameras 无法升级固件来支持这种修改，这就需要引入额外的边缘计算设备来进行预处理，增加了部署成本。

3.2 改进思路

基于上述局限性，可以提出以下改进思路：

1. 目前的参数是静态的。可以引入一个轻量级的、运行在 TEE 内部的预分析模块，根据历史视频流的统计特征动态调整参数。虽然参数调整本身可能泄露长期统计信息，但可以设计差分隐私机制来掩盖具体的参数更新，从而在性能和安全性之间取得更灵活的平衡。

2. 目前的模糊操作是基于软件指令组合实现的。如果未来的 CPU/GPU 架构能提供硬件级别的模糊数据移动指令或对小规模 ORAM 的硬件加速，将能进一

步大幅降低 Visor 的开销。

3. 对于对抗样本检测，在管道前端引入茫然的异常检测模块，识别试图触发最坏情况执行路径的恶意帧，并对其进行降级服务或丢弃，以防御基于资源耗尽的攻击。

3.2. 总结

Visor 是一项具有里程碑意义的工作，它首次展示了在云端构建端到端、高性能且具有强隐私保护的视觉分析服务的可能性。

本文的核心价值在于打破了“隐私保护必定导致极低性能”的固有印象。通过深入分析视频分析算法的内在逻辑，Visor 作者并没有盲目套用通用的安全编译器，而是手工重构了每一个关键的视觉模块。这种领域特定的算法优化——例如将解码与前缀树遍历解耦、将包围盒检测并行化、将裁剪过程两阶段化——使得 Visor 在保证数学上严格的数据茫然性的同时，将性能开销控制在商业可接受的范围内。

此外，Visor 提出的混合 TEE 架构有效地利用了云端异构计算资源，解决了 CPU 与 GPU 之间可信通信的难题。尽管系统仍存在对参数设置的依赖和一定的资源冗余，但它为未来的机密计算应用，特别是涉及大数据量、复杂流水线的应用，提供了一套极具参考价值的设计范式和算法库。Visor 证明了，通过软硬件协同设计和算法创新，由于侧信道攻击带来的“隐私税”是可以被大幅降低的。

参考文献

- [1] A. Ahmad, B. Joe, Y. Xiao, Y. Zhang, I. Shin, and B. Lee. Obfuscuro: A Commodity Obfuscation Engine on Intel SGX. In NDSS, 2019.
- [2] Amazon Rekognition. <https://aws.amazon.com/rekognition/>.
- [3] G. Ananthanarayanan, V. Bahl, P. Bodík, K. Chintalapudi, M. Philipose, L. R. Sivalingam, and S. Sinha. Real-time Video Analytics – the killer app for edge computing. IEEE Computer, 2017.
- [4] Attestation Service for Intel SGX.
<https://api.trustedservices.intel.com/documents/sgx-attestation-api-spec.pdf>.
- [5] J. Bankoski, P. Wilkins, and Y. Xu. Technical overview of VP8, an open source video codec for the web. In ICME, 2011.
- [6] K. E. Batcher. Sorting Networks and Their Applications. In Proceedings of the Spring Joint Computer Conference, 1968.
- [7] B. Bond, C. Hawblitzel, M. Kapritsos, K. R. M. Leino, J. R. Lorch, B. Parno, A. Rane, S. Setty, and L. Thompson. Vale: Verifying High-Performance Cryptographic Assembly Code. In USENIX Security, 2017.
- [8] T. Bourgeat, I. Lebedev, A. Wright, S. Zhang, Arvind, and S. Devadas. MI6: Secure Enclaves in a Speculative Out-of-Order Processor. In MICRO, 2019.
- [9] T. Bouwmans, F. E. Baf, and B. Vachon. Background Modeling using Mixture of Gaussians for Foreground Detection – A Survey. Recent Patents on Computer Science, 2008.
- [10] M. Brandenburger, C. Cachin, M. Lorenz, and R. Kapitza. Rollback and Forking Detection for Trusted Execution Environments using Lightweight Collective Memory. In DSN, 2017.
- [11] F. Brasser, S. Capkun, A. Dmitrienko, T. Frassetto, K. Kostiaainen, and A.-R. Sadeghi. DR.SGX: Automated and Adjustable Side-Channel Protection for SGX Using Data Location Randomization. In ACSAC, 2019.
- [12] F. Brasser, U. Müller, A. Dmitrienko, K. Kostiaainen, S. Capkun, and A. Sadeghi. Software Grand Exposure: SGX Cache Attacks Are Practical. In WOOT, 2017.

- [13] J. V. Bulck, M. Minkin, O. Weisse, D. Genkin, B. Kasikci, F. Piessens, M. Silberstein, T. F. Wenisch, Y. Yarom, and R. Strackx. Foreshadow: Extracting the Keys to the Intel SGX Kingdom with Transient Out-of-Order Execution. In *USENIX Security*, 2018.
- [14] J. V. Bulck, N. Weichbrodt, R. Kapitza, F. Piessens, and R. Strackx. Telling Your Secrets without Page Faults: Stealthy Page Table-Based Attacks on Enclaved Execution. In *USENIX Security*, 2017.
- [15] C. Canella, D. Genkin, L. Giner, D. Gruss, M. Lipp, M. Minkin, D. Moghimi, F. Piessens, M. Schwarz, B. Sunar, J. Van Bulck, and Y. Yarom. Fallout: Leaking Data on Meltdown-resistant CPUs. In *CCS*, 2019.
- [16] S. Cauligi, G. Soeller, B. Johannesmeyer, F. Brown, R. S. Wahby, J. Renner, B. Grégoire, G. Barthe, R. Jhala, and D. Stefan. FaCT: A DSL for Timing-Sensitive Computation. In *PLDI*, 2019.
- [17] G. Chen, S. Chen, Y. Xiao, Y. Zhang, Z. Lin, and T. H. Lai. SgxPectre Attacks: Stealing Intel Secrets from SGX Enclaves via Speculative Execution. In *EuroS&P*, 2019.
- [18] G. Chen, W. Wang, T. Chen, S. Chen, Y. Zhang, X. Wang, and T.-H. L. D. Lin. Racing in Hyperspace: Closing Hyper-Threading Side Channels on SGX with Contrived Data Races. In *IEEE S&P*, 2018.
- [19] S. Chen, X. Zhang, M. K. Reiter, and Y. Zhang. Detecting Privileged Side-Channel Attacks in Shielded Execution with Déjà Vu. In *AsiaCCS*, 2017.
- [20] F. Dall, G. D. Micheli, T. Eisenbarth, D. Genkin, N. Heninger, A. Moghimi, and Y. Yarom. CacheQuote: Efficiently Recovering Long-term Secrets of SGX EPID via Cache Attacks. In *CHES*, 2018.
- [21] N. Dowlin, R. Gilad-Bachrach, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing. CryptoNets: Applying Neural Networks to Encrypted Data with High Throughput and Accuracy. In *ICML*, 2016.
- [22] D. Eppstein, M. T. Goodrich, and R. Tamassia. Privacy-preserving Data-oblivious Geometric Algorithms for Geographic Data. In *Proceedings of the 18th SIGSPATIAL International Conference on Advances in Geographic Information*

Systems (GIS), 2010.

[23] ETSI White Paper No. 11. Mobile Edge Computing – A key technology towards 5G. https://www.etsi.org/images/files/ETSIWhitePapers/etsi_wp11_mec_a_key_technology_towards_5g.pdf.

[24] FFmpeg. <https://ffmpeg.org/>.

[25] M. Fredrikson, S. Jha, and T. Ristenpart. Model Inversion Attacks That Exploit Confidence Information and Basic Countermeasures. In CCS, 2015.

[26] M. Fredrikson, E. Lantz, S. Jha, S. Lin, D. Page, and T. Ristenpart. Privacy in Pharmacogenetics: An End-to-end Case Study of Personalized Warfarin Dosing. In USENIX Security, 2014.

[27] O. Goldreich. The Foundations of Cryptography - Volume 2: Basic Techniques. Cambridge University Press, 2004.

[28] O. Goldreich and R. Ostrovsky. Software Protection and Simulation on Oblivious RAMs. J. ACM, 1996.

[29] J. Götzfried, M. Eckert, S. Schinzel, and T. Müller. Cache Attacks on Intel SGX. In EuroSec, 2017.